Weekly Meeting

Idioms: A Simple and Effective Framework for Turbo-Charging Local Neural Decompilation with Well-Defined Types

Luke Dramko, Claire Le Goues, Edward J. Schwartz
Carnegie Mellon University
Published in NDSS 2026

SecAI Lab
SUNG KYUN KWAN UNIVERSITY

Background: The Decompilation Problem

- **Decompilation**: reconstructing source code from executables
  - Essential for malware analysis, vulnerability research, legacy software
  - Compilation is lossy: variable names, type info are lost

- **Traditional (deterministic) decompilers** (e.g., Hex-Rays, Ghidra)
  - Syntactically correct output but lack semantic details
  - Missing: meaningful variable names, type definitions, struct layouts

- **Neural decompilers**: use LLMs to statistically fill in lost details
  - Predict original source code from decompiler output
  - Key limitation: cannot define user-defined composite types (UDTs)
  - Existing work: LLM4Decompile [Tan'24], Nova [Jiang'25]

Motivation: The UDT Problem in Neural Decompilation

- **User-Defined Types (UDTs)** are pervasive in real-world C code (structs, unions)
  - Essential for understanding memory layout and function semantics
  - No existing neural decompiler can predict UDT definitions!

- **Existing benchmarks mask the problem**
  - HumanEval-Decompile: 0% UDT variables, type complexity = 1.0
  - ExeBench: 2.7% UDT variables, type complexity = 1.3
  - **RealType (new): 15.0% UDT variables, type complexity = 2.5**

- **Two key insights driving Idioms:**
  1. Joint prediction of code + UDT definitions ensures consistency
  2. Scattered evidence problem: neighboring context from call graph needed

Why UDTs Matter: Decompilation Example (Fig. 1)

(a) Original Source Code

```
typedef struct {
  int x;
  int y;
  char* name;
} Point;

void print_point(
    Point* p) {
  printf("%s: (%d,%d)",
    p->name,
    p->x,
    p->y);
}
```

(b) LLM4Decompile (No UDTs)

```
void sub_401000(
    __int64 a1) {
  printf("%s: (%d,%d)",
    *(char**)(a1+8),
    *(int*)a1,
    *(int*)(a1+4));
}

// No type definitions
// Memory offsets only
// Hard to understand
```

Prior work

(c) IDIOMS (Joint Code + UDTs)

```
typedef struct {
  int x;
  int y;
  char* name;
} Point;

void print_point(
    Point* p) {
  printf("%s: (%d,%d)",
    p->name,
    p->x, p->y);
}
```

IDIOMS

Without UDT definitions, decompiled code uses raw memory offsets (a1+8) instead of meaningful field names (p->name).

High-Level Approach: Idioms Framework

- Operates on decompiler output (not raw binary) to leverage deterministic decompilation
- Joint prediction: single model outputs **both source code AND UDT definitions**
- Neighboring context from call graph (BFS) addresses scattered evidence problem
- Fine-tunes pre-trained causal LLMs; QLoRA adapters for >1B models; model-agnostic

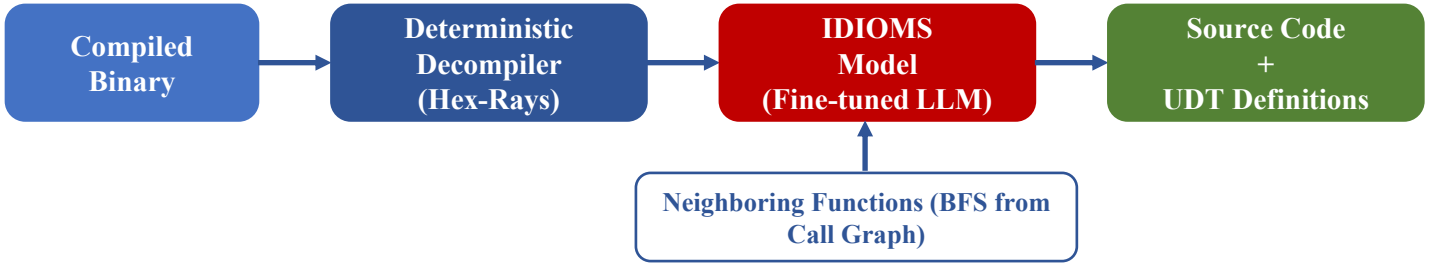
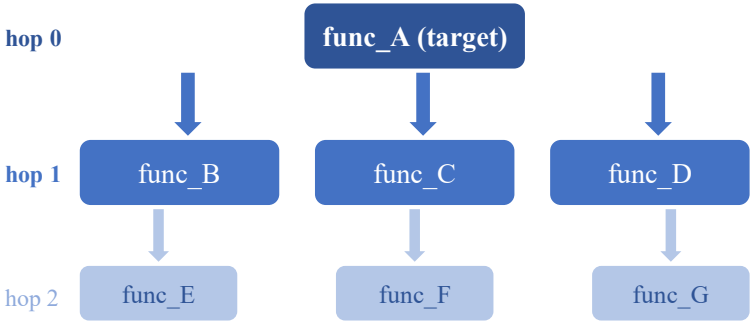


Fig. 2: High-level IDIOMS pipeline (adapted from paper)

Key Insight: Scattered Evidence & Neighboring Context

- **Problem**: A single function only uses a subset of a struct's fields → incomplete picture of the UDT
- **Solution**: Feed neighboring functions (BFS from call graph) into the model as additional context

Call Graph (BFS Traversal)



Scattered Evidence: Partial Field Access

struct config_t { int mode; char *name; float threshold; int flags; }

Function	mode	name	threshold	flags
func_A	✓	✓	✗	✗
func_B	✗	✗	✓	✓
func_C	✗	✓	✗	✓
Combined	✓	✓	✓	✓

Key Result: Neighboring context improves UDT prediction by 63%

RealType Dataset & Data Processing

- **Why a new dataset?**
  - Existing datasets lack (1) UDT definitions and (2) interprocedural call graphs
  - ExeBench: only 2.7% UDT vars, synthetic type defs with "dummy" fields

- **RealType dataset:**
  - 154,301 training functions + 2,862 evaluation functions
  - Complete UDT definitions extracted from preprocessed source code
  - Interprocedural call graphs for neighboring context

- **Data processing pipeline:**
  - Canonical type normalization (e.g., int32_t → int)
  - Deduplication to prevent data leakage across open-source projects
  - Decompilation via Hex-Rays (IDA) and Ghidra

Dataset Comparison (Table I)

Comparison of neural decompilation datasets shows RealType has far more realistic UDTs:

Dataset	# Functions	% UDT Vars	Type Complexity	Call Graphs
HumanEval-Decompile	164	0%	1.0	No
ExeBench	2,383,839	2.7%	1.3	No
RealType (ours)	**154,301**	**15.0%**	**2.5**	**Yes**

- Type complexity = avg. number of nodes in type's tree representation (primitives are leaves)
- RealType: sourced from open-source C projects on GitHub, compiled with gcc, decompiled with Hex-Rays
- **Key insight**: RealType has 5.5x more UDT variables and 2.5x higher type complexity than ExeBench

Evaluation Setup & Metrics

- **Baselines:**
  - LLM4Decompile (1.3B params, Ghidra-decompiled input)
  - Nova (assembly input)
  - Note: SLaDe/DecLLM excluded (require test cases during prediction - unrealistic)

- **Evaluation metrics:**
  - Dependency equivalence: static structural equivalence checking
  - Dep-equiv + types: code equivalence with correct UDT types
  - Variable name accuracy, UDT names + composition accuracy
  - Edit similarity (token-level closeness to ground truth)

- **Research questions:**
  - RQ1: How does Idioms compare to prior work?
  - RQ2: Impact of compiler optimization levels?
  - RQ3: Effect of dataset, context type, and model size (ablation)?

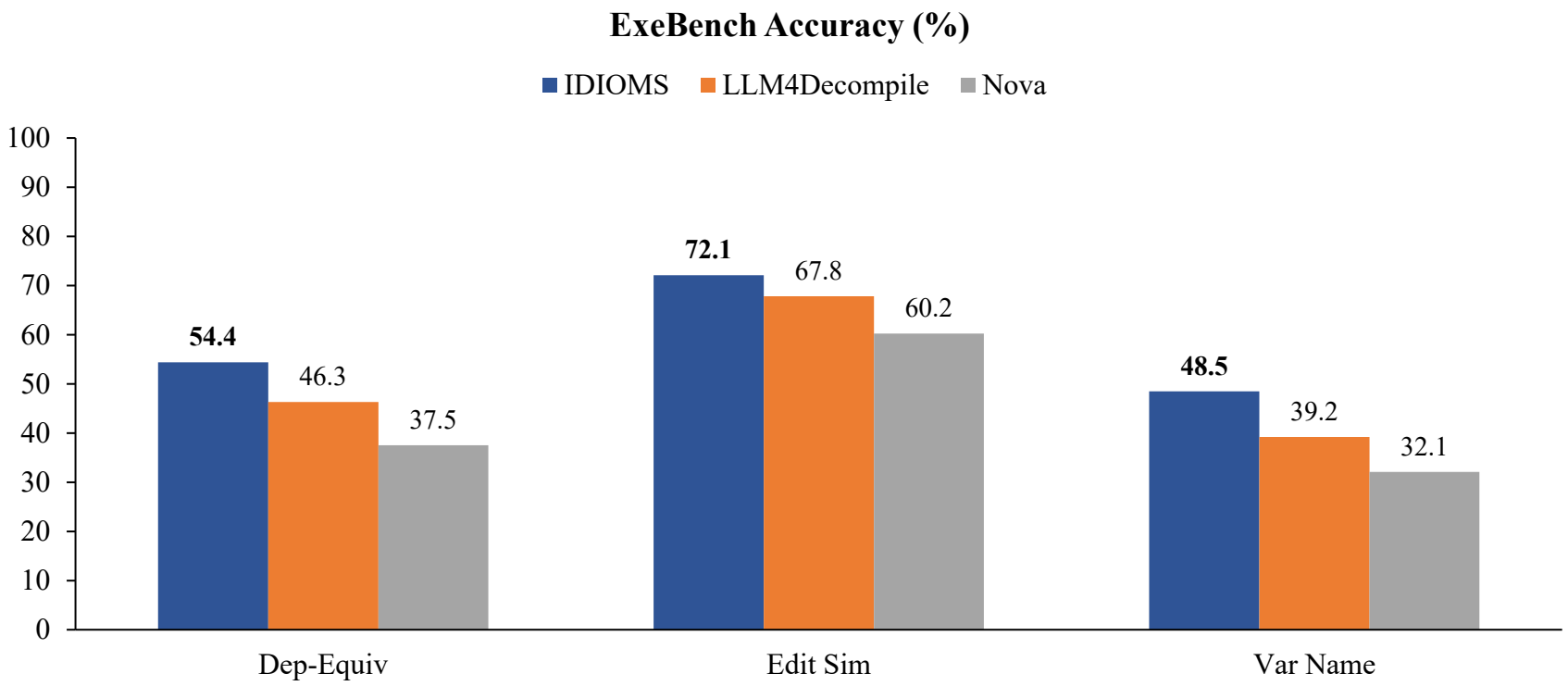
Results: RQ1 - Overall Performance (Table II)

- **On ExeBench (most challenging existing benchmark):**
  - IDIOMS: **54.4%** accuracy vs. LLM4Decompile 46.3% vs. Nova 37.5%
  - 17-36% higher than LLM4Decompile on all correctness metrics
  - 38-78% better than Nova on all correctness metrics

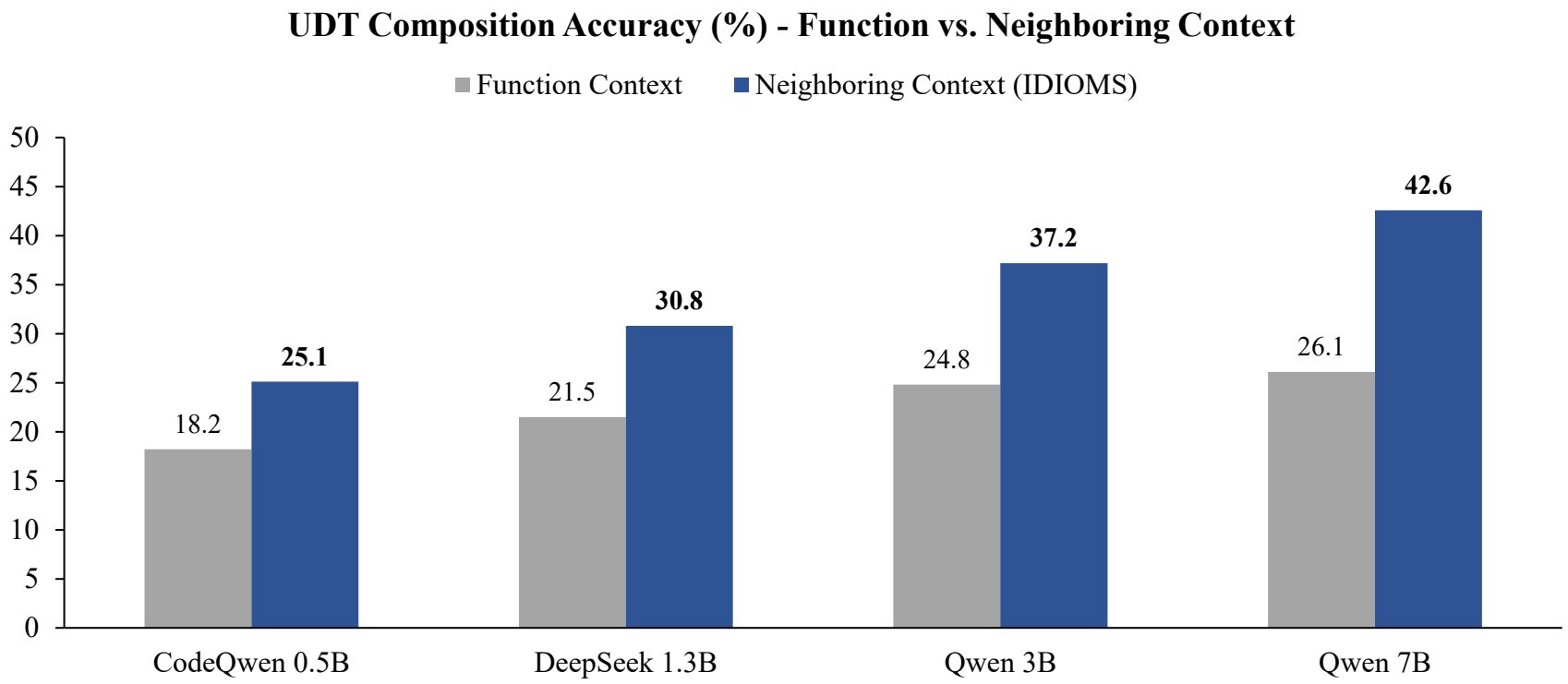
- **On RealType (realistic code with UDTs):**
  - IDIOMS performs **at least 95% better** than baselines
  - Up to 95-205% improvement on realistic code with UDTs
  - Nova/LLM4Decompile score 0% on UDT metrics (they cannot predict UDTs)

- **RQ2 - Compiler optimizations (O0-O3):**
  - IDIOMS consistently outperforms baselines across all optimization levels
  - Inlining disabled to maintain evaluation rigor on RealType

Results: ExeBench Performance Comparison



RQ3: Impact of Neighboring Context on UDT Accuracy



RQ3: Ablation Study (Table III)

- **4 ablation configurations (controlled comparisons):**
- 1. **exebench**: trained on ExeBench (2.4M funcs), function context only
- 2. **parity-exebench**: subsampled ExeBench (same size as RealType), function context
- 3. **functions-realtype**: RealType dataset (154K funcs), function context only
- 4. **neighbors-realtype (IDIOMS)**: RealType, neighboring context (BFS call graph)

Comparison	Finding	Impact
exebench vs parity-exebench	Dataset size (15x) effect	Only 6-11% gain
parity-exebench vs functions-realtype	UDT complexity effect	38-42% drop in dep-equiv
functions-realtype vs neighbors-realtype	Neighboring context effect	Up to 63% UDT improvement

Discussion & Limitations

- **Why does neighboring context help?**
  - UDT fields are scattered across functions (e.g., func A uses .x, func B uses .name)
  - Mirrors how traditional type inference algorithms work: inherently interprocedural
  - Larger models benefit more from neighboring context (up to 63% UDT improvement)
- **Impact of UDT complexity on difficulty:**
  - Switching from ExeBench to RealType causes 38-42% drop in dep-equiv
  - 55-68% drop when type correctness is factored in (dep-equiv + types)
  - Operations on UDTs decompose into complex assembly (e.g., foo->bar->baz)
- **Limitations & future work:**
  - Dataset size (15x larger ExeBench) gives only 6-11% gain → quality > quantity
  - Context window limits how many neighbors can be included
  - Evaluation with Ghidra (free alternative to Hex-Rays) would broaden applicability
  - Integration with separate type recovery tools (e.g., retypd) remains unexplored

Critical Analysis: Strengths & Weaknesses

Strengths

- **Identifies a real, overlooked gap**
  - UDT prediction was genuinely ignored by prior work; the paper convincingly demonstrates that existing benchmarks mask this problem
- **Elegant, simple design philosophy**
  - No architectural changes to the base model; just smart input construction (neighboring context + joint output format) makes this highly reproducible
- **Rigorous experimental design**
  - Thorough ablation study isolates each contribution; tests across 4 model sizes (1B-16B) strengthen generalizability claims
- **Practical scattered evidence insight**
  - BFS-based neighboring context is a transferable idea applicable beyond decompilation (e.g., type inference, program repair)
- **Open science commitment**
  - Dataset, models, and training code released on Zenodo; enables independent verification and follow-up research

Weaknesses

- **C-only, x86-64 only scope**
  - Real-world RE involves C++/Rust/Go with vtables, generics, traits; generalization is unproven
- **Heavy reliance on Hex-Rays**
  - Refines Hex-Rays output, not raw binary; inherits a proprietary tool dependency and its error propagation
- **No semantic correctness guarantee**
  - dep-equiv checks compilation, not behavioral equivalence; predicted UDTs may compile but have wrong semantics
- **Context window bottleneck**
  - Neighbors compete for tokens; long functions truncated, BFS depth limited by sequence length
- **Evaluation gaps**
  - No comparison with type recovery tools (DIRTY, OSPREY); only -O0 tested, rare in real malware

Summary & Key Contributions

- **Contribution 1: RealType Dataset**
  - 154K+ functions with realistic UDTs and complete definitions
  - Exposes that UDTs make decompilation significantly harder
- **Contribution 2: Joint Code + UDT Prediction Training**
  - First neural decompiler to jointly generate code AND type definitions
  - Neighboring context (BFS call graph) improves UDT prediction by up to 63%
- **Contribution 3: State-of-the-Art Results**
  - ExeBench: 54.4% (vs. 46.3% LLM4Decompile, 37.5% Nova)
  - RealType: 95-205% better than baselines on code with UDTs
- **Contribution 4: Open Science**
  - Publicly released: RealType dataset, Idioms models, code (github.com/squaresLab/idioms)

SecAI Lab

Thank you